

AI-Assisted Software Development

Advanced Code Generation Using Dual Models

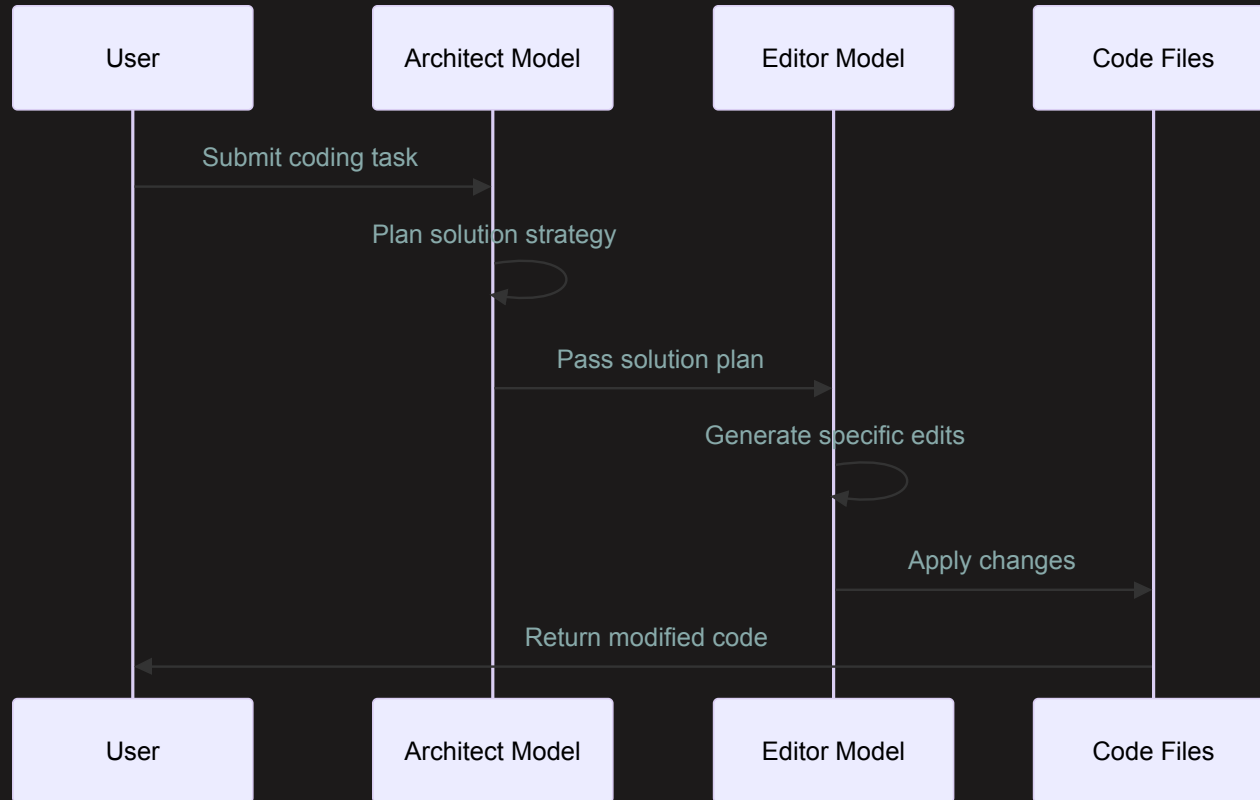
Mike Burton 4-6 November 2025

What is Architect Mode?

- New experimental feature in Aider
- Uses two AI models for enhanced code generation:
 - Architect model for planning
 - Editor model for implementation
- Introduces a planning stage before code changes
- Aims for improved accuracy and efficiency

How Architect Mode Works

Architect Mode interacts with two models:



Two-Model Approach

In the two-model approach, one model acts as architect while the other acts as implementer:

Architect Model:

- Plans the solution
- Defines strategy
- Creates high-level design
- Considers architecture

Editor Model:

- Implements the plan
- Generates specific code
- Makes precise edits
- Maintains consistency

Key Features

Here are some of the key features of Aider:

1. Dual Model System

- Architect for planning
- Editor for implementation

2. Smart Defaults

- Automatic editor selection
- Based on architect model

3. Customization Options

- Override default editor
- Choose specific combinations

4. Simple Integration

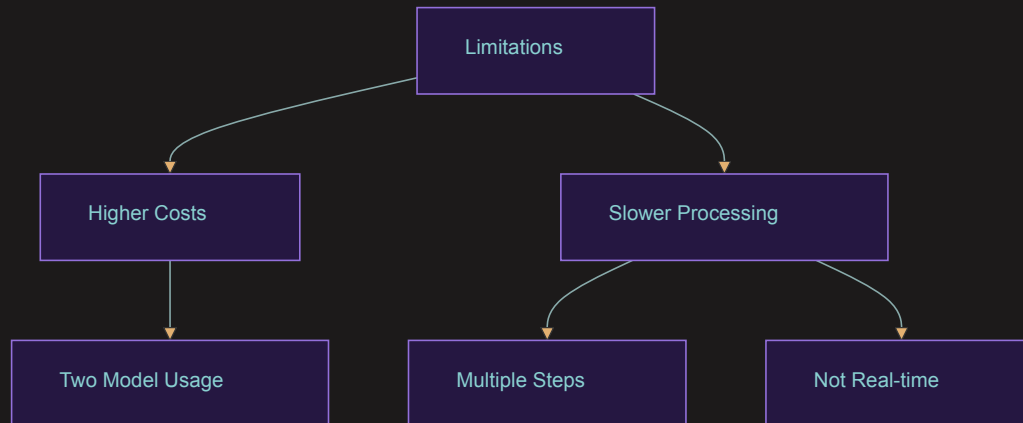
- Use `/architect` command
- Switch modes easily

Benefits vs Limitations

Benefits:

- Improved accuracy
- Better planning
- Structured approach
- Enhanced efficiency

Limitations:



Recommended Combinations

These model combinations have been found to work well:
(Check for newer versions)

Best Overall (Whole Edit Format):

Architect: o1 Preview

Editor: DeepSeek

Practical Option (Diff Edit Format):

Architect: o1 Preview

Editor: Claude 3.5 Sonnet

Good Performance (Diff Edit Format):

Architect: DeepSeek

Editor: GPT-4

Using Architect Mode

1. Installation:

```
# Install development version of Aider  
# (See optional steps page)
```

2. Basic Usage:

```
model: o1 Preview  
architect
```

3. Custom Editor:

```
model: o1 Preview  
architect  
editor model: Claude 3.5 Sonnet
```


Best Practices

1. Choose the Right Combination:

- Consider task complexity
- Balance speed vs accuracy
- Account for cost constraints

2. Planning Phase:

- Be clear about requirements
- Provide context
- Specify constraints

3. Implementation:

- Review generated plan
- Verify changes
- Test results

When to Use Architect Mode

Best For:

- Complex refactoring
- New feature design
- Architecture changes
- Large-scale updates

Maybe Not For:

- Simple edits
- Quick fixes
- Real-time collaboration
- Cost-sensitive tasks

Tips for Success

1. Clear Requirements:

- Be specific
- Provide context
- State constraints

2. Model Selection:

- Match to task
- Consider trade-offs
- Test combinations

3. Review Process:

- Check planning output
- Verify implementations
- Test thoroughly

Example Workflow

1. Start Architecture Session:

```
/architect
```

2. Define Task:

```
Create a user authentication system with:
```

- JWT tokens
- Role-based access
- Password encryption

3. Review Plan

4. Implementation

5. Verify Results

Recap: Architect Mode

- Two-model approach
- Enhanced planning
- Improved accuracy
- Customizable setup
- Trade-offs to consider
- Best practices to follow

Next Steps

- Experiment with combinations
- Test on different tasks
- Measure results
- Optimize workflow

Questions to Consider:

- Which tasks benefit most?
- What combinations work best?
- How to optimize costs?